План реализации минимально жизнеспособной версии «Ева 2.0»

Описание проекта: *«Ева 2.0» – это интеллектуальный ИИ-агент с модульной архитектурой, способный к самонаблюдению и* ***самоизменению*** *собственного кода. Он сочетает в себе "мозг" (ядро принятия решений и память) и внешнюю LLM-модель (например, GPT-4) для генерации идей и кода. Цель минимально жизнеспособной версии – продемонстрировать полный цикл: получение задачи → планирование → генерация кода → тестирование → анализ ошибок → исправление кода → сохранение результатов → извлечение уроков.*

Этап 1: Настройка окружения и инструментов

На первом этапе подготавливается базовое окружение для работы Евы 2.0: устанавливаются необходимые программы, библиотеки и обеспечивается доступ к языковой модели. Эти шаги создадут прочный фундамент для последующей разработки.

Шаг 1.1: Установка Python и необходимых библиотек

- **Действия:** Установить интерпретатор Python 3.x и создать изолированное виртуальное окружение (например, с помощью `venv`), чтобы управлять зависимостями проекта. Затем установить ключевые библиотеки:
    - Клиент для доступа к API GPT-4 (например, официальный SDK OpenAI) – потребуется для отправки запросов к языковой модели.
    - Библиотеку для работы с PostgreSQL (например, `psycopg2` или SQLAlchemy) – для взаимодействия с базой данных, которая послужит «памятью» Евы.
    - Инструменты для Docker (например, `docker` Python SDK) – для управления контейнерами песочницы.
    - Библиотеки для анализа кода (встроенный модуль `ast`, возможно `radon` для метрик) – чтобы Ева могла парсить свой код и вычислять метрики.
- **Цель:** Обеспечить наличие всех технических средств, необходимых Еве для генерации и исполнения кода, хранения данных и изолированного тестирования.
- **Метрика успеха:** Проверить, что окружение настроено корректно – **успешный импорт** всех необходимых пакетов в Python. Косвенный показатель – выполнение пробного запроса к GPT-4 (ожидается валидный ответ). Это подтвердит работоспособность связки с моделью. Также можно измерить *время импорта библиотек* и *время ответа* простого GPT-запроса, чтобы убедиться в приемлемой скорости.

Шаг 1.2: Настройка доступа к GPT-4 (LLM)

- **Действия:** Получить API-ключ для GPT-4 (или другой выбранной LLM) и сохранить его безопасно (например, в переменной окружения). Создать модуль `llm_proxy.py` – это прокси-обёртка для взаимодействия с языковой моделью. Он будет принимать текстовые промпты от Евы и возвращать ответы модели. Внутри него реализовать функции для отправки запросов к OpenAI API (с учётом контроля ошибок и таймаутов).
- **Цель:** Инкапсулировать детали работы с внешней LLM, чтобы DecisionEngine мог просто отправлять запросы через этот модуль. **LLM Proxy** станет “голосом” Евы – благодаря нему система сможет понимать естественный язык и генерировать текст/код по запросу.
- **Метрика:** *Время отклика и точность ответов модели.* Измерять среднее **время ответа API** (например, 1–2 секунды для простых запросов) и отслеживать **расход токенов**. Также вести счет успешных ответов: в скольких случаях сгенерированный код сразу проходит тесты или оказывается полезным без дополнительной правки. Если GPT-4 слишком медленный или дорогой – эти метрики подскажут, когда стоит оптимизировать промпты или перейти на более быструю модель.

Шаг 1.3: Установка PostgreSQL и настройка базы «памяти»

- **Действия:** Установить СУБД PostgreSQL локально. Создать новую базу данных (например, `eva_memory`). Подключить расширение **pgvector** (если планируется хранение эмбеддингов для семантического поиска – на будущее). Настроить таблицы для хранения ключевых данных Евы:
  - `modules` (или `code_map`) – мета-карта кода: описание всех «кубиков» (модулей) Евы, их связей и метрик.
  - `tasks` – задачи/цели, полученные Евой.
  - `plans` – планы выполнения задач.
  - `code_versions` – разные версии сгенерированного кода и патчей.
  - `test_results` – результаты прогонов тестов (успешность, ошибки).
  - `reflections` – журнал рефлексии (опыт: что пробовала, какие уроки вынесла). *(На этом этапе можно заложить упрощенную структуру, добавляя детали по мере надобности на следующих этапах.)*
- **Цель:** Организовать долговременную память для Евы. Вся структура кода, знания о прошлых попытках и улучшениях будут надежно храниться и быстро извлекаться через SQL-запросы. **Централизация** в БД обеспечивает целостность: вместо разрозненных YAML-файлов все ключевые данные находятся в одном месте, что упрощает управление связями и версионирование.
- **Метрика:** *Надёжность и быстродействие памяти.* Измерять **время выполнения запросов** к БД (например, <50 мс для основных запросов). Также контролировать **актуальность данных** – процент записей, которые устарели или неверны, должен быть ~0% (в MVP все записи свежие, далее механизм очистки устаревших знаний может быть добавлен). Если запросы памяти

начинают занимать слишком много времени или выдавать некорректные данные, это сигнал улучшить индексы или логику обновления памяти.

Шаг 1.4: Установка Docker и подготовка песочницы

- **Действия:** Установить Docker Engine на машину. Проверить, что docker-команды выполняются без ошибок. Настроить образ Docker (например, на основе python:3.X) для запуска сгенерированного кода: он должен включать Python и нужные пакеты. На этапе MVP можно использовать стандартный python-образ и устанавливать зависимости перед каждым тестом.
- **Цель:** Создать **изолированную среду** («лабораторию») для безопасного исполнения кода Евы. Это защитит основную систему: если сгенерированный код содержит ошибки, бесконечные циклы или потенциально опасные операции, они не повлияют на “боевую” систему, выполняясь внутри контейнера.
- **Метрика:** *Корректность работы песочницы.* Простой тест: запуск контейнера с выполнением тривиального скрипта (`print("Hello")`) должен проходить успешно. Замерить **время запуска контейнера** и выполнения скрипта (например, несколько секунд). В дальнейшем метриками надежности песочницы станут: процент случаев, когда сбой внутри контейнера **не влияет** на хост (ожидается 100%), и среднее **время подготовки песочницы** под тесты (стремиться минимизировать, чтобы не тормозить цикл обучения Евы).

Шаг 1.5: Создание каркаса проекта и структурирование модулей

- **Действия:** Определить структуру директорий и файлов для кода Евы 2.0. Разделить проект на смысловые части (по аналогии с «органами»):
  - **Корневая директория `eva2.0/`** – содержит основной запускаемый файл (например, `run_eva.py` или `main.py`), а также конфигурации (можно добавить `README.md`, `requirements.txt`).
  - **`eva_core/`** – ядро Евы. В этой папке разместить ключевые модули «мышления» и координации:
    - `decision_engine.py` – центральный модуль (мозг), о нём подробнее в Этапе 2.
    - `memory_manager.py` – модуль работы с памятью (PostgreSQL).
    - `self_monitor.py` – модуль самонаблюдения (отслеживание собственной структуры и состояния).
    - `improvement_planner.py` – модуль планирования улучшений (анализа опыта).
    - `feedback_requester.py` – модуль для запросов обратной связи (например, к пользователю) при необходимости.
    - *(Другие базовые “кубики” можно добавить по мере реализации следующих этапов.)*
  - **`eva_code/`** – модули генерации и редактирования кода (условно “руки”):
    - `code_generator.py` – генерация нового кода (функций, классов) по описанию.

        - `code_editor.py` – точечное редактирование существующего кода (внесение правок, рефакторинг).
        - `code_checker.py` – верификация кода (может быть объединён с тестами или самонаблюдением).
    - **`eva_test/`** – модули тестирования (условно “лаборатория”):
        - `test_generator.py` – генерация тестовых сценариев для кода.
        - `test_runner.py` – запуск тестов внутри Docker-песочницы.
        - `test_analyzer.py` – анализ результатов тестов, определение причин сбоев.
        - `sandbox_manager.py` – утилиты для управления Docker-контейнером (создать/запустить/удалить).
    - **`eva_utils/`** – вспомогательные утилиты:
        - `state_manager.py` – управление состоянием системы (например, флаги «АКТУАЛЬНО» / «НА ИЗМЕНЕНИИ» для модулей).
        - `code_map.py` – (может быть частью self_monitor) логика сбора структуры кода (мета-карты).
        - `extract_libraries.py`, `install_libraries.py`, `cleanup_libraries.py` – работа с зависимостями (получение списка требуемых пакетов, установка, очистка).
        - `llm_proxy.py` – уже созданный модуль доступа к LLM (из шага 1.2).
        - и др. утилиты (логирование, конфиг).
    - **`eva_api/`** – (опционально на MVP) веб- или CLI-интерфейс:
        - `api.py` – FastAPI endpoints, если нужен веб-доступ.
        - `cli.py` – интерфейс командной строки.
        - (На этапе MVP можно начать с простого CLI или вообще прямого запуска без сложного интерфейса).
- **Цель:** Убедиться, что код структурирован модульно. **Каждый файл соответствует отдельному “кубику” функциональности**, что отражает принцип высокой связности внутри модуля и слабой связанности между модулями. Такая структура облегчит дальнейшую разработку: параллельную работу над разными частями и замену/обновление модулей без влияния на другие.
- **Метрика:** *Завершённость архитектурного каркаса.* Можно ввести показатель процентной готовности: например, N из M запланированных модулей созданы (даже если пустые пока). Цель – 100% модулей имеют заглушки/шаблоны к концу этапа. Это количественно отражает прогресс. Качественная метрика – **понятность структуры**: новый разработчик, взглянув на папки и названия файлов, должен без труда понять основные компоненты системы (можно провести ревью или опрос).

Примечание: На данном этапе мы не пишем подробную логику в этих модулях, а лишь определяем места под неё. Это как строительство здания: мы заложили фундамент (Python, GPT-4, PostgreSQL, Docker) и возвели каркас (директории и файлы под

каждую функцию системы). Далее будем «заполнять» этот каркас логикой шаг за шагом.

Этап 2: Архитектура системы и основные компоненты

Теперь, имея подготовленное окружение, создаём **концептуальную архитектуру** Евы 2.0 – определяем ключевые модули (кубики) и их роли, а также то, как они взаимодействуют друг с другом. На этом этапе важно заложить **принцип "синапсов"** – связи между модулями, через которые будет передаваться управление и данные. Модули должны общаться через четко определённые интерфейсы (например, функции с известными входами/выходами), что имитирует нейронные связи: один кубик окончил работу и передает результат следующему, **не напрямую вызывая конкретный код, а через протокол взаимодействия**.

Основные компоненты архитектуры Евы 2.0 и их предназначение:

Шаг 2.1: Ядро принятия решений (Decision Engine) – файл `eva_core/decision_engine.py`

- **Роль:** Это «мозг» Евы. DecisionEngine отвечает за глобальный контроль: получает входные цели/задачи и решает, как их выполнить. Он разбивает задачу на подзадачи, выбирает нужные модули-исполнители и оркестрирует их выполнение по порядку (как диспетчер). В терминах синапсов – DecisionEngine находится в центре сети, направляя сигналы по нужным связям: **получив задачу → обращается к памяти за контекстом → вызывает генератор кода → инициирует тесты → анализирует результаты → решает, достигнута ли цель** или требуется новая итерация.
- **Вход:** Высокоуровневая цель или задача (от пользователя *либо внутренняя*, например, от ImprovementPlanner из Этапа 7). Также получает данные о текущем состоянии системы: метрики производительности, карту кода, результаты предыдущих операций.
- **Выход:** Команды для других модулей (например: «сгенерируй функцию X», «запусти тесты», «скорректируй код») и финальный ответ/результат, возвращаемый пользователю (если задача внешняя).
- **Связи:**
    - Активно взаимодействует с **MemoryManager** (для получения контекста, сохранения новых знаний).
    - Вызывает **LLM Proxy** для творческих задач (генерация плана, кода или объяснений).
    - Запускает цепочки модулей в **eva_code** и **eva_test** (генерация кода, тестирование, улучшение).
    - Общается с **SelfMonitor** для обновления знаний о себе (например, после изменений кода).

- **Метрика:** *Успешность выполнения задач.* Можно измерять долю целей, которые система смогла решить, а также **среднее число итераций цикла** “Цель → План → Действие → Оценка → Урок” до достижения успеха. Идеально, если большинство задач решаются с 1–2 циклами. Если DecisionEngine часто попадает в тупики или требует много попыток, это сигнал доработать алгоритмы планирования и выбора действий.

Шаг 2.2: Прокси для LLM (LLM Proxy) – файл `llm_proxy.py`

- **Роль:** Модуль-посредник между “мозгом” Евы и внешней языковой моделью GPT-4. DecisionEngine формирует запрос (промпт) – LLM Proxy передаёт его через API и возвращает ответ модели. Таким образом, LLM Proxy предоставляет Еве “расчётный интеллект”: способность понимать естественный язык и генерировать осмысленный текст или код.
- **Действия:** Инкапсулирует детали API-вызова: формирует правильный формат запроса, отправляет и получает результат. Например, DecisionEngine может попросить: «Составь план решения задачи X», LLM Proxy отправит этот текст GPT-4 и вернёт сформулированный план.
- **Интерфейс:** Предоставляет методы вроде `ask_model(prompt, context)` → возвращает строку-ответ. Благодаря этому DecisionEngine не зависит от конкретной модели – можно подменить GPT-4 на другую реализацию, изменив только LLM Proxy.
- **Метрика:** *Качество взаимодействия с моделью.* Отслеживать **точность и полезность ответов**: в скольких случаях ответ LLM подходит без доработки (например, сгенерированный код сразу проходит тесты). Также мониторить **время и токены**: средняя задержка ответа и стоимость. Эта метрика позволит балансировать качество/скорость – например, для простых запросов можно переключаться на более лёгкую модель, если GPT-4 избыточен.

Шаг 2.3: Менеджер памяти (MemoryManager) – файл `eva_core/memory_manager.py`

- **Роль:** Центральное хранилище знаний Евы. MemoryManager предоставляет интерфейс для чтения и записи данных “памяти” – тех самых таблиц PostgreSQL, настроенных на этапе 1. Он отвечает за хранение:
  - **Мета-карты кода** – структурного описания всех модулей (кубиков) Евы: какие файлы и функции существуют, как они связаны (импорты, вызовы), какие метрики качества у них.
  - **Фактов и истории** – например, какие изменения вносились, какие баги находились и исправлялись (ReflectionLog).
  - **Целей и планов** – сохранённые задания, планы их выполнения, результаты (чтобы Ева помнила, что она делала раньше).
- **Действия:** MemoryManager предлагает методы вроде `get_module_info(name)` или `save_test_result(task_id, result)`. Внутри он решает, где хранятся эти данные – в SQL таблицах, JSONB-полях или YAML-файлах. На этапе MVP можно многие вещи хранить прямо в PostgreSQL. **Пример использования:** DecisionEngine спрашивает: «Дай описание модуля X из мета-карты» – MemoryManager выполняет SQL-запрос к таблице modules и

возвращает сохранённое описание. Или DecisionEngine говорит MemoryManager: «Сохрани новый факт: тест Y пройден» – и модуль вставляет запись в таблицу тестовых результатов.

- **Связи:** DecisionEngine обращается к MemoryManager при планировании (получить контекст о предыдущих решениях), SelfMonitor обновляет через него данные о коде, ImprovementPlanner читает из памяти метрики и историю для анализа.
- **Метрика:** *Надёжность и быстродействие памяти.* Ключевой показатель – **время доступа к памяти**: например, чтение необходимого факта не должно замедлять цикл работы (стремиться к миллисекундам). Другой показатель – **актуальность данных**: если MemoryManager возвращает устаревшую или неправильную информацию, DecisionEngine примет неверные решения. Цель – минимизировать расхождения (в идеале, MemoryManager всегда выдаёт свежие данные, так как SelfMonitor будет обновлять мета-карту сразу после изменений). Можно логировать процент случаев, когда информация из памяти оказывается неактуальной, и стремиться к 0%.

Шаг 2.4: Мониторинг собственного состояния (SelfMonitor) – файл `eva_core/self_monitor.py`

- **Роль:** Этот модуль даёт Еве базовое **самосознание**: знание о своей собственной структуре и состоянии. SelfMonitor следит за «физическим состоянием» кода Евы – какие модули есть, что в них находится, насколько они эффективно работают. Прообразом служит функциональность `code_map.py` из версии 1.0: там карта кода строилась для одного файла, здесь нужно охватить всю кодовую базу.
- **Действия:**
  - **Сканирование кода:** SelfMonitor периодически (или по запросу DecisionEngine) рекурсивно проходит по директориям проекта, **читает все файлы .py** (и др., как конфиги), **парсит их в AST** (abstract syntax tree) и извлекает ключевые элементы – функции, классы, импорты.
  - **Обновление мета-карты:** На основе этого анализа SelfMonitor формирует/обновляет записи в базе (через MemoryManager) – фактически поддерживает актуальную таблицу `modules` (или `code_map`). Например, если добавлен новый файл или функция, SelfMonitor добавит о нём запись; если изменён код – пересчитает метрики (сложность, длина и пр.).
  - **Сбор метрик:** SelfMonitor может вызывать подсистему **MetricCollector** (можно оформить как часть его или отдельный модуль) для подсчёта статических метрик качества кода: цикломатическая сложность каждой функции, количество строк, уровень связности, покрытие тестами и т.д.. Эти метрики также сохраняются в мета-карте.
- **Связи:** SelfMonitor тесно связан с MemoryManager – после каждого изменения в коде DecisionEngine может вызывать SelfMonitor, чтобы тот пересканировал изменённый модуль и внёс обновления. Также SelfMonitor предоставляет DecisionEngine информацию: «в системе сейчас N модулей, из них M – в рабочем состоянии, такие-то имеют ошибки». Он сотрудничает с TestEngine и

CodeEditor: после тестов узнаёт, прошли ли они, и помечает модуль здоровым или проблемным.

- **Метрика:** *Полнота и актуальность самонаблюдения.* В идеале SelfMonitor должен описывать **100% кода** Евы в своей мета-карте. Показатель – процент файлов/функций, присутствующих в карте. Если что-то не отражено, значит система «слепа» к этой части себя. Ещё показатель – **задержка обновления:** время между изменением кода и обновлением мета-карты (лучше обновлять сразу после изменений). И наконец, **оценка здоровья кода:** SelfMonitor может сравнивать текущие метрики модулей с эталонными (например, сложность ниже 10, покрытие тестами 90%+) и сигнализировать, насколько текущий код от этих целей отклоняется.

Шаг 2.5: Планировщик улучшений (ImprovementPlanner) – файл `eva_core/improvement_planner.py`

- **Роль:** Этот модуль – «внутренний аналитик» Евы. Он регулярно просматривает информацию о работе системы (метрики из SelfMonitor, журнал ReflectionLog) и выявляет **возможности для самооптимизации**. Проще говоря, ImprovementPlanner решает, *что* Еве стоило бы улучшить в себе самой далее.
- **Действия:**
  - **Анализ ошибок:** Изучает записи в журнале рефлексии (о нём – в Этапе 7): какие ошибки часто повторяются? Например, если видит, что за последнюю неделю 3 раза возникала ошибка «ImportError», делает вывод: *нужно улучшить механизм управления библиотеками*.
  - **Анализ метрик:** Отслеживает динамику метрик кода. Если, например, средняя цикломатическая сложность функций растёт, ставит цель: *провести рефакторинг для упрощения кода*. Или видит, что покрытие тестами низкое в некоторых модулях – цель: *дописать тесты для них*.
  - **Формирование задач улучшения:** Генерирует понятные задачи для DecisionEngine. Например: «улучшить модуль X, снизив его сложность с 20 до 10» или «добавить обработку ошибки Y в функцию Z». Эти задачи он может ставить в очередь целей DecisionEngine, который затем будет их выполнять, проходя цикл самосовершенствования.
  - **Приоритизация:** Если потенциальных улучшений много, ImprovementPlanner расставляет приоритеты – на MVP достаточно простого подхода: что чаще вызывает проблемы, то важнее. Например, если 5 раз были проблемы с модулем A и 1 раз с модулем B, сначала берётся за A.
- **Связи:** ImprovementPlanner получает данные от **SelfMonitor** (метрики текущего кода) и читает **ReflectionLog** (из памяти через MemoryManager). Вывод своей работы – *новые внутренние цели* – он передаёт в DecisionEngine (в виде обновления списка задач или прямого вызова, как будто пользователь поставил задачу, только от имени системы).
- **Метрика:** *Результативность самоулучшений.* Отслеживать, сколько предложенных ImprovementPlanner улучшений действительно реализовано и к чему привело. Например: из 5 запланированных оптимизаций выполнено 4, и 3 из них дали положительный эффект на метрики (сложность снизилась, скорость повысилась и т.д.). Если большинство планов полезны – отлично; если много

улучшений холостые или вредные, значит, Planner выделяет не то – потребуется доработка алгоритмов анализа.

Шаг 2.6: Модуль обратной связи (FeedbackRequester) – файл `eva_core/feedback_requester.py`

- **Роль:** Вспомогательный модуль, который Ева использует для взаимодействия с человеком, когда ей нужна оценка или подсказка. Хотя цель – максимальная автономность, на этапе MVP полезно предусмотреть «клапан», через который система может запросить помощи.
- **Действия:** Например, если DecisionEngine не уверен, принимать ли рискованное изменение, он может вызывать FeedbackRequester: тот отправит пользователю вопрос вроде «Обнаружено два варианта оптимизации кода: A и B. Какой выбрать?». В простейшем случае, FeedbackRequester может просто логировать подобные сообщения или отправлять их через интерфейс (консоль/веб) и ждать ответа.
- **Связи:** DecisionEngine решает, когда нужна внешняя обратная связь (например, по таймауту решения задачи или при неопределённости). ImprovementPlanner тоже может через этот модуль спрашивать подтверждение на глобальные изменения.
- **Метрика:** *Взаимодействие с пользователем.* Можно считать, сколько раз за определённый период Ева обратилась за помощью – для MVP это должно быть минимально необходимое количество. Если система постоянно спрашивает, значит, она недостаточно автономна. И наоборот, если ни разу не запросила, а должна была (и сделала ошибку) – это тоже показатель. Цель – **сбалансировать автономность и контроль**: Ева решает сама всё, что может, и обращается только по действительно важным поводам.

После выполнения этапов 1–2 у нас будет готова «нервная система» Евы 2.0: основные центры и коммуникации между ними. **Принцип синапсов** заложен: каждый кубик автономно выполняет свою функцию и передаёт управление дальше по определённым связям. Эти связи могут быть явно прописаны (DecisionEngine знает последовательность вызовов) или записаны в **«карту связей»** в базе данных (таблицы, описывающие, какой модуль за каким следует). В MVP достаточно фиксированной логики в коде, но на будущее мы предусмотрели хранение структуры и зависимости модулей в памяти – это позволит Еве в будущем **динамически перестраивать свои цепочки** (добавлять новые кубики или менять последовательность, имитируя образование новых нейронных путей).

Метрика этапа: доля реализованных (или хотя бы спроектированных) ключевых модулей системы. Например, если описаны и созданы файлы всех 5–6 основных компонентов, этап успешен на 100%. Это показатель того, что архитектурный каркас полностью охватывает необходимые функции.

Этап 3: Реализация памяти и самоконтроля (Self-Monitoring)

Теперь, когда архитектура определена, сконцентрируемся на **памяти Евы и механизме самонаблюдения**. Цель этого этапа – дать системе понимание своего собственного кода (построить мета-карту) и возможность сохранять знания о себе. Это фундамент для любых будущих самоизменений: Ева должна видеть, *из чего она состоит*, и хранить эту информацию структурированно.

Шаг 3.1: Реализация мета-карты кода («ментальной карты»)

- **Действия:** Разработать логику, с помощью которой Ева будет собирать информацию о своем коде. В модуле `self_monitor.py` (SelfMonitor) реализовать функцию, которая:
    1. **Сканирует файловую систему проекта:** получает список всех файлов и папок внутри `eva2.0/` (начиная от корня проекта). Включает .py файлы модулей, а также важные файлы конфигурации (YAML, Dockerfile и пр.).
    2. **Парсит каждый файл с кодом:** с помощью `ast` разбирает Python-файлы и извлекает из них структуру: список определённых функций, классов, их методов, глобальных переменных, а также используемые импорты модулей. Важные детали:
        - Имя функции/класса, ее расположение (имя файла, строки начала-конца).
        - Докстринг или комментарий «ОТВЕЧАЕТ ЗА» (если мы документируем назначение кубика в коде) – чтобы получить описание модуля.
        - Сигнатура функции: параметры и возвращаемое значение (можно позже использовать для схемы ввода/вывода кубика).
    3. **Определяет логические «кубики»:** на основе полученных данных решает, какие функции/классы образуют самостоятельные функциональные блоки. (В простом случае, можно считать каждый определённый `def` или класс верхнего уровня – отдельный кубик, а можно пометить особым образом основные модули.) Например, функция `generate_code()` – это отдельный кубик “Генератор кода”.
    4. **Вычисляет связи между кубиками:** анализирует, какие функции вызывают какие (по имени или импортам) – тем самым строит граф зависимостей. Например, если `plan_engine.py` импортирует `llm_proxy` и вызывает `ask_model`, то фиксируется связь: PlanEngine -> LLMProxy. Эти **«синаптические связи»** отражают последовательность логического потока между модулями.
    5. **Собирает статические метрики:** для каждого обнаруженного кубика рассчитывает показатели качества:
        - Размер (число строк кода).
        - Цикломатическая сложность функций.
        - Число исходящих/входящих вызовов (меры связности).

        - Наличие тестов (можно по соглашению: есть ли файл `test_<module>.py`).
        - и т.д. (список метрик можно расширять).
- **Цель:** Получить подробную «картину» текущего состояния Евы 2.0 в машиночитаемом виде.
- **Результат:** Данные мета-карты (список модулей с их свойствами и связями) сохраняются либо во временный YAML-файл (например, `code_map.yaml`), либо **сразу записываются в базу данных** (предпочтительно, в таблицы `modules` и `dependencies`). Для MVP можно сначала вывести в YAML для наглядности, а затем занести в БД.
- **Метрика:** *Полнота мета-карты.* Проверить, что в собранной структуре присутствуют записи обо **всех основных модулях**, которые мы создали на этапе 2. Оценить процент покрытия: например, «отражено 95% файлов, 5% (может, вспомогательных) пропущено». Стремиться к 100%. Кроме того, убедиться, что **связи корректны**: для каждого модуля перечислены действительно вызываемые им другие модули (можно проверить вручную на нескольких примерах или автотестами на основе известной структуры проекта).

Шаг 3.2: Хранение и управление данными памяти

- **Действия:** Теперь необходимо интегрировать SelfMonitor с MemoryManager, чтобы данные мета-карты и другие знания сохранялись долговременно:
    - Расширить структуру таблиц в PostgreSQL для хранения мета-карты. Например:
        - Таблица `modules`: колонки `module_id` (уникальный id), `name` (имя модуля/функции), `description` (короткое описание или docstring), `file_path`, `start_line`, `end_line`, `metrics` (JSON с метриками), `status` (активный/в разработке/с ошибками), `last_modified`.
        - Таблица `dependencies`: колонки `from_module_id`, `to_module_id` (какой модуль вызывает какой), тип связи (импорт, прямой вызов), и пр.
        - Таблица `tests`: колонки о наличии тестов для модуля, процент покрытия.
        - *(Можно сгруппировать: либо одна большая таблица для мета-карты, либо несколько связанных – на усмотрение, но связь «кубик – связи – метрики» должна сохраняться.)*
    - Реализовать в `memory_manager.py` функции для записи результатов SelfMonitor: например, `update_module_record(module_info)` – добавляет или обновляет запись о модуле в таблице, `update_dependency(from, to)` – фиксирует связь.
    - При первом запуске SelfMonitor занести **все данные мета-карты в БД**. Это станет первоначальным «зеркалом» Евы – её знанием о самой себе.
    - Настроить MemoryManager на **извлечение** данных: запросы вида `get_modules(status=None)` (список всех модулей, можно

фильтровать по статусу), `get_module_by_name(name)`, `get_dependents(module_id)` (кто зависит от данного модуля) и т.д.

- **Цель:** Сделать так, чтобы вся информация о структуре кода, связях и статических характеристиках была **легкодоступна** внутри программы. Теперь DecisionEngine или ImprovementPlanner могут в любой момент спросить MemoryManager о любых аспектах устройства Евы.
- **Метрика:** *Быстродействие и актуальность памяти:* После внедрения БД-структуры измерить время выполнения основных операций MemoryManager (например, получение полного списка модулей или всех зависимостей какого-то модуля). Они должны выполняться за считанные миллисекунды при небольшом объёме данных MVP. Также важно, чтобы **после каждого изменения** (см. этап 5) SelfMonitor/MemoryManager обновляли эти записи. Метрикой может быть **количество рассинхронизаций** между реальным кодом и данными памяти (стремиться к 0). Например, если модуль удалён, а в таблице `modules` он остался – это рассинхронизация, её следует избегать.

Шаг 3.3: Контекстная память и ассоциативные связи (опционально)
(Этот шаг более продвинутый, для расширения возможностей памяти, может частично выйти за рамки MVP.)

- **Действия:** Добавить в MemoryManager поддержку семантической памяти:
  - **Рабочая память:** хранение текущего контекста выполнения. Например, текущая задача, промежуточные результаты. Это может быть просто глобальная структура в MemoryManager или отдельная таблица.
  - **Векторные представления знаний:** если подключён pgvector, можно сохранять embedding-ы описаний ошибок, решений, модулей. Впоследствии Ева сможет делать **семантический поиск** – находить похожие проблемы по описанию или вспоминать, где она уже видела похожий код.
  - **Механизм забывания:** внедрить простой *счётчик использования* или метку времени для записей в памяти (например, ReflectionLog может расти, нужно уметь отфильтровывать очень старые данные). Пока можно просто ограничиться небольшим объёмом (MVP не забудет, но по мере развития – да).
- **Цель:** Заложить основы долгосрочной памяти, которая не просто хранит факты, но и связи между ними. Например, если модуль A часто менялся после ошибок, можно увязать эти события – ассоциативно помнить, что «A проблемный».
- **Метрика:** *Объём памяти и скорость поиска.* Например, сколько записей в ReflectionLog или modules хранится и как быстро по ним можно искать. На MVP объёмы малы, но важно убедиться, что подход масштабируем. Метрика – время поиска по ключевым полям (название модуля, тип ошибки) – должно расти линейно с количеством записей и на малых объёмах быть неощутимым.

В результате этого этапа у Евы появляется **внутренняя модель самой себя**. Она знает, какие у неё есть модули, как они связаны и какими свойствами обладают. Реализована требуемая meta-code map – карта кода, лежащая в основе самопонимания. Теперь Ева имеет постоянную память и может использовать её перед принятием решений или при анализе результатов.

Этап 4: Ядро принятия решений и маршрутизация задач

Имея фундамент в виде памяти и самоконтроля, переходим к «поведению» Евы – разработке самого цикла принятия решений. На этом этапе реализуем логику, по которой Ева будет **принимать поступающие задачи и выполнять их**, включая маршрутизацию к нужным модулям. Проще говоря, «оживляем мозг»: теперь DecisionEngine начнёт реально дергать остальные модули, связывая всё вместе.

Шаг 4.1: Обработка входящей задачи (внешней или внутренней)

- **Действия:** Реализовать механизм получения задачи:
  - Если задача приходит от пользователя (например, через API или CLI) – модуль `input_gateway.py` (из структуры этапа 1) принимает её. Предположим, пользователь формулирует задачу на естественном языке или YAML. InputGateway преобразует её в стандартный вид: например, записывает в таблицу `tasks` новую запись со статусом «новая» и основным текстом задачи.
  - Если задача – *внутренняя*, от ImprovementPlanner, она может сразу помещаться в очередь DecisionEngine (минуя внешний интерфейс).
  - В любом случае DecisionEngine получает структуру задачи: описание + возможно контекст (приоритет, тип задачи).
  - После получения новой цели DecisionEngine заносит её в свой рабочий список целей.
- **Логика:** DecisionEngine может поддерживать очередь или стек целей. На MVP достаточно взять по одной задаче последовательно.
- **Метрика:** *Время реакции на новую задачу.* Измерять, сколько времени проходит от поступления задачи до начала её обработки (DecisionEngine начал планировать). Желательно, чтобы задача не «зависала» без внимания (для MVP – мгновенно, т.к. задачи обрабатываются по мере появления). Также можно фиксировать событие в лог: «Новая задача принята: ...».

Шаг 4.2: Планирование решения (с участием LLM)

- **Действия:** Для поступившей задачи DecisionEngine сначала формирует **план решения** – последовательность шагов/подзадач. Это можно сделать, используя мощь GPT-4:
  - DecisionEngine вызывает LLM Proxy с промптом вида: *«Разбей задачу X на план действий: шаги 1,2,3...»*. Получив ответ (например, YAML-план или список шагов), DecisionEngine сохраняет его (в таблицу `plans` или как структуру в памяти).
  - Если план получился неконкретным или слишком общим, DecisionEngine может уточнить его сам или снова спросить LLM (итеративное улучшение

плана). Эту логику можно частично перенять из `task_parser.py` версии 1.0 (где генерировался `task_workplan.yaml`).

- **Логика:** План может включать пункты вроде «Сгенерировать функцию A», «Создать класс B», «Написать тесты для ...», «Запустить код и проверить результат». DecisionEngine позже сопоставит эти пункты с модулями, которые их выполнят.
- **Метрика:** *Качество плана.* Формально измерить сложно, но можно вести статистику: насколько часто первоначальный план оказывается выполнимым без серьёзных корректировок. Например, если 80% задач решаются по исходно сгенерированному плану (пусть и с мелкими исправлениями кода), то планирование работает хорошо. Если часто приходится менять план на ходу – возможно, стоит улучшить промпты или алгоритм планирования.

Шаг 4.3: Маршрутизация подзадач к модулям

- **Действия:** DecisionEngine проходит по плану и для каждого шага определяет, какой модуль должен его исполнить:
  - Например, шаг «Сгенерировать функцию A» → передать описание этой функции (может быть текст из плана) в модуль `code_generator.py`.
  - Шаг «Запустить тесты для модуля X» → вызвать `test_runner.py` с указанием, какие тесты выполнить.
  - Шаг «Проанализировать сбой» → вызвать `test_analyzer.py` или `code_checker.py` в зависимости от природы сбоя.
  - Если планирует улучшение существующего кода → вызвать `code_editor.py` с нужной инструкцией.
- **Логика реализации:** В DecisionEngine можно использовать простой `if/elif` по типам шагов, либо диспетчер таблицу: например, словарь `{"generate_code": call CodeGenerator, "run_tests": call TestRunner, ...}`. На будущее, эти соответствия можно вынести в конфигурацию (в ту же БД: таблицу decision_rules – *IF* задача типа X *THEN* модуль Y).
- **Передача данных:** Важно, что DecisionEngine передает в модуль всю необходимую информацию: описание функции, или ссылку на модуль для тестирования, и т.п. – **четкий интерфейс**. Например, для CodeGenerator: `generate_code(module_name, spec)`, для TestRunner: `run_tests(module_name or file_path)`.
- **Метрика:** *Правильность маршрутизации.* Количественно: доля шагов плана, которые удалось сопоставить с модулем автоматически. На MVP ожидается 100% для известных типов шагов. Если DecisionEngine получает непонятный шаг, которого нет в маршрутизации, это проблема (нужно либо доработать DecisionEngine, либо промпт планирования). Также можно измерять **время на переключение**: DecisionEngine не должен стать узким местом, дергая модули – вызов модуля должен происходить практически мгновенно после завершения предыдущего, если ресурсы позволяют (использовать асинхронность или очереди при необходимости, но MVP можно сделать синхронно).

Шаг 4.4: Выполнение цикла действий и принятие решений об итерациях

- **Действия:** Запустить последовательное выполнение шагов:
    1. DecisionEngine вызывает соответствующий модуль.
    2. Ожидает от него **результат** (например, CodeGenerator возвращает сгенерированный код или указатель на созданный файл).
    3. Анализирует результат: если модуль выполнил шаг успешно (код сгенерирован, тесты прошли), переходит к следующему шагу. Если возникла *ошибка* или *неуспех*, DecisionEngine может принять решение: либо повторить шаг (например, снова попытаться сгенерировать код с другими параметрами), либо отклонить текущий план и сформировать новый, либо – чаще всего – вызвать процедуру обработки ошибки (например, TestAnalyzer).
    4. После прохождения всех шагов плана DecisionEngine оценивает, достигнута ли исходная цель. Если да – помечает задачу завершенной (статус «done» в `tasks` и формирует ответ пользователю). Если нет – может попробовать альтернативный план или выдать сообщение о невозможности решить (для MVP достаточно зафиксировать неудачу).
- **Логика:** Здесь принимаются *решения на основе результатов*. Например, DecisionEngine может иметь простые правила: «Если тесты провалены – вызвать улучшение кода и повторить тесты; если улучшение не помогло за N попыток – спросить совета или откатить изменения». Эти правила можно захардкодить или тоже вынести в таблицу (например, `decision_rules`: condition -> action).
- **Метрика:** *Итеративность цикла.* Вести счётчик итераций для каждой задачи: сколько раз цикл генерация→тест→исправление повторился. Например, задача могла требовать 3 итерации (генерировали код 3 раза, пока тесты не стали зелёными). Метрика – **среднее число итераций**. Если оно растёт слишком высоко, это значит, что либо CodeGenerator недостаточно качественный, либо TestAnalyzer плохо предлагает исправления – зона для улучшения.

Шаг 4.5: Финализация задачи и обучение

- **Действия:** По завершении задачи DecisionEngine:
    - Формирует **отчёт**: что сделано, какой результат (успех или описание неудачи). Для пользователя это может быть текстовый вывод с пояснением. Также сохраняет этот результат в `ReflectionLog` (журнал) – «Задача X выполнена, заняло Y итераций, полученный код Z, уроки ...».
    - Отмечает в памяти: обновляет статус записи задачи (например, `tasks.status = 'done'`), сохраняет финальный код в `code_versions` (как новую версию).
    - Если были временные файлы или контейнеры – инициирует их очистку (например, SandboxManager уничтожает Docker-контейнер).
    - Проверяет очередь целей: если есть ещё задачи (или ImprovementPlanner что-то добавил) – переходит к следующей.
- **Логика:** На этом шаге можно также дать команду SelfMonitor еще раз пересканировать изменённый код (чтобы мета-карта зафиксировала новое состояние после всех правок) – хотя SelfMonitor лучше запускать

непосредственно после каждого изменения, повторное сканирование в конце не повредит для консистентности.

- **Метрика:** *Качество конечного решения.* Например, **процент успешных задач** из всех попыток. Пока задачи искусственные (мы сами даём), но можно отслеживать: если что-то не удалось – разбор причин. Также **время решения задачи** end-to-end – от момента поступления до отчёта – характеризует эффективность. В MVP ожидаемо минуты (с учётом запросов к GPT и тестов); со временем должно снижаться.

Архитектура принятия решений, реализованная на этом этапе, обеспечивает организованное выполнение задач. Ева 2.0 теперь не просто знает о себе, но и умеет действовать по заданной цели, координируя свои модули. **Принцип гибридного интеллекта** проявляется здесь: DecisionEngine и внутренняя логика – это “символическое сознание”, а GPT-4 через LLM Proxy – “нейросетевой интеллект” для творческих шагов. Важно, что мы чётко разграничили: нейросеть используется только когда нужно придумать решение или код, а вся критическая логика (что делать и когда) остаётся прозрачной и управляемой – это повышает надёжность системы.

Этап 5: Реализация модулей генерации и изменения кода (*«кубики-руки»*)

На данном этапе создаём инструменты, с помощью которых Ева может **создавать новый код и модифицировать свой собственный код**. Если предыдущие этапы дали Еве «глаза» (видеть свой код) и «мозг» (принимать решения), то теперь мы даём ей **«руки»** – способность действовать в материальном плане кода: писать и переписывать программы. Это критически важно для самомодификации.

Шаг 5.1: Генерация нового кода (модуль CodeGenerator) – файл `eva_code/code_generator.py`

- **Действия:** Реализовать функцию генерации кода с помощью LLM:
  - Функция `generate_code(spec)` принимает спецификацию того, что нужно реализовать. Например, текст описания функции или класса, либо структура из плана (название функции, список требований).
  - Внутри формируется промпт для GPT-4: можно использовать шаблон, включающий описание задачи, может быть часть существующего кода (чтобы модель знала контекст). Пример промпта: *«Напиши реализацию функции `calculate_factorial(n)` на Python. Требования: рекурсивный алгоритм, обработка отрицательных входов... Current codebase info: (список импортов, если нужно).»*
  - Отправить запрос через `llm_proxy` и получить сгенерированный фрагмент кода (текст).
  - Сохранить этот код: либо как файл (например, создать новый `.py` или добавить в существующий), либо хотя бы вернуть его DecisionEngine. На

MVP можно записывать сразу в файл, чтобы потом запустить тесты на нём.
    - Использовать утилиты: можно задействовать `extract_libraries.py` – после генерации кода вытащить, какие библиотеки нужно импортировать, и подготовить их (возможно, установить через pip, но лучше собрать список).
- **Логика:** CodeGenerator должен уметь генерировать разные фрагменты:
    - **Новый модуль целиком:** если план предусматривает создание нового файла/модуля.
    - **Отдельную функцию или метод:** вставить в уже существующий класс.
    - **Фрагмент кода**: например, реализацию части логики.
    - В MVP достаточно генерации простой функции или небольшого класса по описанию.
- **Интерфейс с DecisionEngine:** CodeGenerator возвращает указание, что сделано – например, имя файла, в который записан код, или сам текст кода (DecisionEngine может потом его сохранить).
- **Метрика:** *Качество генерируемого кода.* Основной показатель – **проходят ли тесты** сгенерированного кода с первого раза. Например, метрика: % кода, прошедшего все тесты без исправлений. Также учитывать **размер и сложность кода**: GPT-4 может сгенерировать громоздкий код, поэтому хорошо, если код соответствует требованиям и не избыточен (сложно формализовать, но можно следить за метриками сложности: не превышает оговоренный порог).

Шаг 5.2: Точечное редактирование кода (модуль CodeEditor) – файл `eva_code/code_editor.py`

- **Действия:** Реализовать функции, позволяющие вносить **изменения в уже существующий код**:
    - `modify_function(module, func_name, new_code)`: заменяет тело функции `func_name` в указанном модуле на `new_code`.
    - `add_method(class_name, method_code)`: добавляет новый метод в класс.
    - `insert_import(module, import_line)`: добавляет строку импорта, если её нет.
    - `delete_lines(module, start, end)`: удаляет определённые строки (например, удалить устаревшую функцию).
    - *Можно реализовать как отдельные функции или одну универсальную, которая применяет «патч» изменений.*
    - **Парсинг AST:** Для точности лучше не делать замену простым поиском/строкой, а использовать AST. Например, Python модуль `ast` и `astor` для обратно генерации: прочитать модуль → найти нужный узел (функцию) → заменить узел → записать обратно код. На MVP, однако, можно и регулярными выражениями или простым текстовым поиском, если формат кода предсказуем.
- **Логика:** CodeEditor – как хирург: он не генерирует с нуля, а исправляет. Команды на исправление могут приходить от DecisionEngine на основе анализа:

например, «увеличить таймаут в функции X с 5 до 10 секунд» – CodeEditor находит в коде X место, где задан таймаут, и правит значение.
    - Также возможен сценарий: DecisionEngine получил от TestAnalyzer рекомендацию «Добавить проверку на None в функцию Y» – CodeEditor должен уметь это сделать. Возможно, проще опять обратиться к GPT-4, но уже с контекстом кода и конкретной инструкцией (то есть CodeEditor сам по себе тоже может использовать LLM, как продвинутая версия).
- **Интерфейс:** CodeEditor можно соединить с LLM Proxy, чтобы генерировать патчи. Но для MVP можно часть изменений формировать алгоритмически (например, для типовых ошибок).
- **Метрика:** *Точность изменений.* Измерять, сколько раз внесённая правка **ломает другой функционал**. В идеале, изменение локальное и не затрагивает ничего постороннего – это проверяется тестами: если после исправления возникли новые ошибки, значит вмешательство повлекло побочные эффекты. Цель – минимизировать такие случаи. Ещё метрика – **количество возвратов**: сколько раз одну и ту же вещь приходилось исправлять повторно. Если часто – возможно, CodeEditor не до конца выполняет задачу с первого раза.

Шаг 5.3: Проверка и валидация кода (модуль CodeChecker) – файл `eva_code/code_checker.py`

- **Действия:** Этот модуль служит для статической проверки качеств кода до или параллельно с тестами:
    - Может запускать **линтер** (например, `flake8`/`pylint`) на сгенерированный код и парсить его вывод, выявляя потенциальные проблемы (неиспользуемые переменные, синтаксические недочёты).
    - Может проверять **соответствие структуры**: например, сравнить, что сгенерированный код содержит функции, перечисленные в плане (вдруг GPT-4 что-то лишнее придумал или пропустил).
    - **Валидация контрактов:** если мы для каждого кубика определили входы/выходы (например, через Pydantic-схемы), CodeChecker мог бы проверить, что новый код этим контрактам соответствует (но это больше на будущее).
- **Логика:** CodeChecker работает как «контроль качества» перед тестированием. В MVP достаточно, например, парсить синтаксические ошибки, которые может допустить GPT. Однако GPT-4 редко делает синтаксические ошибки, скорее логические – их линтер не увидит.
    - Можно сосредоточиться на проверке стиля и очевидных проблем.
    - Также CodeChecker может **автоматически формировать небольшие тесты** (например, если видит функцию без теста, создать шаблонный тест-вызов).
- **Интерфейс:** DecisionEngine может вызывать CodeChecker сразу после генерации кода, чтобы быстро получить фидбэк, не дожидаясь полноценных тестов.
- **Метрика:** *Количество выявленных проблем статически.* Например, из 10 ошибок, приведших к сбоям тестов, сколько могли бы быть обнаружены линтером/статическим анализом. Если значительная часть – усиление

CodeChecker улучшит эффективность (поймали проблему раньше – сэкономили время).

Шаг 5.4: Интеграция генерации/правок с мета-картой и памятью

- **Действия:** Встроить вызовы SelfMonitor/MemoryManager в процессы генерации и изменения:
  - После того, как CodeGenerator создал новый код (или CodeEditor изменил существующий), **SelfMonitor вызывается на обновлённый файл** – чтобы добавить/обновить запись в мета-карте и пересчитать метрики.
  - MemoryManager помечает модуль как “в черновике” или “требует тестирования” (например, статус в таблице modules = ‘modified’).
  - DecisionEngine знает, что этот код пока *не подтверждён тестами*, и дальше, при успешных тестах, отметит его активным.
  - Также целесообразно использовать систему **версий кода**: перед изменением можно сохранять старую версию в таблицу `code_versions` (или в Git-репозиторий, если подключим). Это позволит откатиться, если изменение окажется неудачным.
- **Цель:** Обеспечить согласованность: Ева изменила код – Ева сразу же осознала, что изменилась. Таким образом, не будет ситуации, что память говорит одно, а реальный код другое.
- **Метрика:** *Актуальность мета-карты после изменений.* Измерять временной лаг между изменением и обновлением записи. Например, сделать тест: добавить функцию через CodeEditor и проверить, обновилась ли мета-карта (MemoryManager) до запуска следующего шага. Метрика – процент изменений, корректно отражённых сразу же (хотим 100%).

Завершив этот этап, мы наделили Еву 2.0 способностью **менять свой код**. Теперь у неё есть «руки», которыми она может как строить новые части программы, так и переделывать существующие. В сочетании с предыдущими этапами (памятью и мозгом) это значит, что Ева может замкнуть цикл: *понять задачу → изменить себя для решения задачи → проверить, получилось ли*. Однако без тщательной проверки есть риск “сломать себя”. Поэтому следующий этап – создать безопасную **лабораторию** для экспериментов, где Ева будет обкатывать свои изменения.

Этап 6: Тестирование и безопасная песочница (*«лаборатория»*)

Для способности к самосовершенствованию критично, чтобы Ева могла **проверить любые изменения** в своем коде, прежде чем «впустить» их в основную работу. Этап 6 посвящён созданию механизма, позволяющего Еве запускать свои (изменённые или новые) компоненты в изолированном окружении, собирать результаты и анализировать ошибки.

Шаг 6.1: Генерация тестов (модуль TestGenerator) – файл `eva_test/test_generator.py`

- **Действия:** Разработать функциональность автоматического создания тестовых сценариев для сгенерированного или изменённого кода:
  - Функция `generate_tests(module_name, spec)` – на вход получает имя модуля/функции и (опционально) спецификацию или описание её поведения.
  - Формирует запрос к GPT-4 (через LLM Proxy) с просьбой написать **юнит-тесты** для данного кода. Можно передать сам код функции в промпте или хотя бы описание, какие случаи покрыть.
  - Получает от модели код тестов (например, класс с `unittest` или функции с `pytest`).
  - Сохраняет тесты во временный файл, например, `test_<module_name>.py`.
  - Также может использовать заранее подготовленные шаблоны: например, у нас есть `test_templates.py` с заготовками на типовые случаи (из Ева 1.0) – их тоже можно задействовать для консистентности стиля.
- **Логика:** TestGenerator можно сначала сделать простым: генерировать 1–2 базовых теста проверяющих основную функциональность. На будущее – генерировать и граничные случаи, и нештатные (исключения).
  - Важный момент: если функция принимает вход, GPT-4 не знает конкретных значений, которые правильны. Возможно, стоит передавать и ожидаемый результат (если он известен). Но часто задачи формулируются так, что можно придумать проверку. Например, если функция “возводит число в квадрат”, понятно, какие входы проверить.
- **Интерфейс:** DecisionEngine вызывает TestGenerator сразу после CodeGenerator или CodeEditor, чтобы подготовить тесты перед запуском.
- **Метрика:** *Покрытие и адекватность тестов.* Показатель – **процент кода, покрытого тестами**. Для MVP цель – хотя бы ключевые функции покрыты (можно измерять строками или функциями). Также субъективный критерий – тесты действительно отлавливают ошибки: если код неверен, тест падает. Увидим это по тому, как часто тесты проходят сразу. Если код неправильный, а тесты тоже проходят – значит, тесты слабые. Стремиться к тому, чтобы **каждая функциональность имела хотя бы один тест**, а лучше несколько разных.

Шаг 6.2: Запуск тестов в Docker-песочнице (модуль TestRunner) – файл `eva_test/test_runner.py`

- **Действия:** Реализовать механизм изолированного запуска сгенерированного кода и тестов:
  - При вызове `run_tests(target_module)` модуль использует **SandboxManager** (`sandbox_manager.py`) для подготовки контейнера:
    - Собирает текущий код Евы (всю папку или только нужный модуль и зависимости) и копирует/монтирует его внутрь Docker-контейнера.

        - Устанавливает необходимые библиотеки: можно установить заранее известные (`requirements.txt` проекта) и любые дополнительные, выявленные через `extract_libraries`.
        - Запускает команду внутри контейнера, например: `pytest -q test_<module>.py` (или python -m unittest).
    - Собирает результаты выполнения: возвращает лог тестов, список пройденных/упавших тестов, сообщения об ошибках/traceback.
    - По завершении тестирования контейнер уничтожается (или сохраняется для отладки, но в MVP проще удалять, чтобы не скапливались).
- **Логика:** TestRunner должен обеспечить, что даже если внутри тест запускается бесконечный цикл или происходит сбой, это не повлияет на основную систему:
    - Можно задать таймауты: например, если тест не завершился за X секунд – принудительно остановить контейнер и считать тест проваленным.
    - Ограничить ресурсы контейнера (CPU/RAM) – Docker это позволяет – чтобы сбойный код не съел все ресурсы.
- **Интерфейс:** DecisionEngine или ImprovementPlanner вызывает TestRunner, передавая информацию, что тестировать. Например, после генерации функции `foo` DecisionEngine: `TestRunner.run_tests("foo")`.
- **Метрика:** *Надёжность песочницы.* Ключевой показатель – **изолированность**: после запуска кода в контейнере состояние основной системы не изменилось (кроме полученных логов). Это можно неявно проверить: например, после выполнения тестов сравнить хеш файлов проекта – он должен совпадать с исходным (ни один файл не изменён посторонним кодом). Также важна **скорость развертывания тестов** – на MVP это десятки секунд (с учётом запуска Docker). В дальнейшем можно оптимизировать (кешировать образ). Метрика – среднее время цикла “запустить контейнер + выполнить тесты” и доля случаев, когда тесты завершились успешно vs сбоем vs по таймауту.

Шаг 6.3: Анализ результатов тестирования (модуль TestAnalyzer) – файл `eva_test/test_analyzer.py`

- **Действия:** Обрабатывать вывод тестов и выявлять причины ошибок:
    - Функция `analyze_results(test_log)` на вход получает лог/результат выполнения тестов от TestRunner.
    - Если **все тесты прошли**, то всё отлично: возвращает статус OK.
    - Если есть **ошибки**, парсит сообщения об ошибках. Например, по traceback может определить тип: ошибка импорта (ImportError – значит, библиотека не найдена), ошибка AssertionError (сам тест не прошёл – значит, логика не соответствует ожиданиям), SyntaxError или NameError (проблемы в коде) и т.д.
    - На основе типа ошибки может:
        - *Ошибка импорта:* предложить через ImprovementPlanner добавить нужную библиотеку (или проверить orthography).
        - *AssertionError:* значит, код не удовлетворяет требованию – возможно, логика неполная.
        - *Ошибка во время выполнения:* например, IndexError, KeyError – что-то не учли.

        - Для каждой категории можно прописать рекомендацию.
    - **Использование LLM:** Для сложного анализа можно обратиться к GPT-4: передать ему текст ошибки и контекст, спросить «что пошло не так и как это исправить?». GPT-4 может вернуть подсказку, которую TestAnalyzer структурирует в предложение (например: "Function X не учитывает отрицательный аргумент, нужно добавить проверку").
    - Возвращает **диагноз** – описание проблемы и, при возможности, предложение, как исправить.
- **Логика:** На MVP можно сделать упрощённо: анализировать по ключевым словам. Например, если в логе есть "AssertionError: expected 10 but got 12", то понять, что логика выдала неправильный результат – ImprovementPlanner может поставить цель улучшить точность. Если "TypeError: ..." – значит неправильный тип где-то, можно предложить добавить проверку типов.
- **Связи:** DecisionEngine получает от TestAnalyzer вывод диагностики и на его основе решает, что делать:
    - Если есть конкретное предложение фиксить – передаёт его в CodeEditor (шаг 5.2) для применения.
    - Если неясно – может привлечь FeedbackRequester (спросить у человека, что делать).
- **Метрика:** *Точность диагностики.* Из всех падений тестов, в скольких система правильно определила причину. Это оценивать сложнее, но на тестовых задачах можно заложить известные ошибки и смотреть, поймет ли TestAnalyzer. Также метрика – **количество циклов генерация→тест→фиксы**, как упоминалось: если много повторных неудач, возможно, анализ был неверный и дал неправильный совет.

После этапа 6 у Евы есть полноценная **лаборатория**. Теперь она способна в автономном режиме проверять любые свои изменения и только при успешном исходе внедрять их в основную кодовую базу. Это замыкает практически полный цикл: от задачи до результата с самопроверкой. Ева может безопасно экспериментировать со своим кодом без страха мгновенно «сломаться» – все риски локализованы внутри Docker.

Метрики этапа:

- **Покрытие тестами**: желаемо, чтобы каждое изменение сопровождалось тестом. Показатель – сколько новых строк кода *в среднем* проверяются хотя бы одним тестом.
- **Отсутствие побочных эффектов**: после выполнения контейнера убедиться, что никаких лишних файлов или процессов не осталось. Это индикатор, что изоляция работает.
- **Скорость цикла тестирования**: важна, чтобы цикл улучшения не сильно тормозился. Если на каждый прогон уходит, скажем, 1 минута, а требуется 5 итераций, то 5 минут – терпимо. Если гораздо больше, стоит оптимизировать.

Этап 7: Петля самообучения и самоулучшения (*Self-Feedback Loop*)

На данном этапе соединяем воедино механизмы рефлексии и планирования улучшений, созданные ранее, чтобы Ева могла **учиться на своём опыте**. Здесь реализуется настоящее саморазвитие: на основе накопленных данных о своей работе система будет корректировать своё поведение, предлагать новые цели и совершенствовать навыки без внешней подсказки.

Шаг 7.1: Ведение журнала рефлексии (ReflectionLog)

- **Действия:** Убедиться, что на протяжении всех предыдущих этапов Ева фиксировала ключевые события в журнале:
  - Формат можно сделать YAML или записывать в таблицу `reflections` в БД. Каждая запись содержит:
    - Время события.
    - Описание действия или этапа (например: «Сгенерирован код для функции X», «Тесты для X провалены»).
    - Результат (успех/неуспех, кратко).
    - **Причина неудачи**, если есть (например: AssertionError, или "вышло время").
    - **Урок**, если выучен (что мы извлекли).
  - На MVP первые три пункта обязательно, причина и урок – опционально, будем добавлять по мере возможности.
  - Пример записи: `[12:00] Задача: оптимизировать функцию X. Действие: сгенерирован код v2. Результат: тесты провалены (AssertionError). Урок: нужно учитывать граничные случаи.`.
  - **Автоматизация уроков:** Можно попытаться сразу формировать урок через LLM: после провала DecisionEngine формирует вопрос LLM типа «Что можно вынести из этой ситуации?». GPT-4 ответит, например: "Нужно обработать случай пустого списка". Этот текст сохранять как lesson. Если пока сложно, ImprovementPlanner позже сам выведет уроки, но место под них заложить.
  - Хранить лог либо в памяти (БД) либо файл `reflection_log.yaml`. Важно, чтобы ImprovementPlanner мог делать по нему запросы – значит, лучше БД (таблица `reflections`).
- **Цель:** Создать долговременную память об опыте. Если память modules/metrics – это как Ева знает свое тело, то ReflectionLog – как она помнит свои прошлые *события жизни*.
- **Метрика:** *Полнота и точность журнала.* Стремиться записывать **100% критически важных событий**: каждое изменение кода, каждый запуск тестов, каждое получение новой цели. Если что-то не записано, часть памяти теряется. Точность – записи должны соответствовать реальности. Проверка: анализируя журнал, ImprovementPlanner делает правильные выводы. Если в логе путаница (например, неправильное время или перепутаны причины), Planner будет ошибаться – значит, журнал ведётся плохо.

Шаг 7.2: Анализ журнала и метрик (улучшение через ImprovementPlanner)

- **Действия:** Теперь, когда ReflectionLog и метрики (SelfMonitor) собирают информацию, настроить ImprovementPlanner (из этапа 2) на их регулярный анализ:
    - **Автоматический запуск:** Можно вызывать ImprovementPlanner:
        - Периодически (например, раз в день/раз в N задач).
        - Или по событию (например, сразу после завершения каждой задачи – тогда он оценивает, не пора ли что-то улучшить).
    - **Анализ шаблонов ошибок:** ImprovementPlanner просматривает последние N записей журнала:
        - Какие типы ошибок встречаются часто? (Например: “в 3 задачах из 5 возникали проблемы с импортами”).
        - Какие модули чаще всего изменялись из-за ошибок? (Если модуль A исправляли 4 раза, а B ни разу – A проблемный).
    - **Анализ динамики метрик:** Смотрит хранящиеся метрики из SelfMonitor:
        - Растёт ли суммарная сложность кода с течением времени? (Если да – код усложняется, нужно планировать рефакторинг).
        - Как меняется покрытие тестами? (Если падает – возможно, добавляем код без тестов, надо исправить).
        - Как меняется среднее время выполнения тестов? (Если увеличивается – код становится медленнее, можно оптимизировать).
    - **Выявление приоритетных проблем:** Допустим, обнаружено:
        - Слишком высокая сложность функции X.
        - Часто падают тесты, связанные с модулем Y.
        - Низкое покрытие тестами в области Z. ImprovementPlanner решает, что важнее (например, по частоте или влиянию на другие модули).
    - **Формирование целей улучшения:** Для каждой проблемы создаётся улучшенческая задача: например, *«Уменьшить цикломатическую сложность модуля X на 20%»*, *«Добавить обработку ошибок в функцию Y»*, *«Написать дополнительные тесты для Z»*. Эти задачи ставятся во внутреннюю очередь DecisionEngine (либо сразу передаются ему через MemoryManager). Возможно, некоторые задачи можно объединить или упорядочить (сначала поправить то, что мешает остальному).
    - **Инкрементальность:** ImprovementPlanner не должен перегружать систему кучей улучшений сразу. Пусть выдаёт 1–2 цели, дождётся их выполнения, потом переоценит состояние. Это предотвратит конфликтующие изменения и даст время учиться. (На MVP можно просто 1 цель за раз.)
    - **Учет успехов:** Анализирует не только проблемы, но и успешные решения – чтобы закрепить их как лучшие практики. Например, видит, что добавление проверки на None в модуле X полностью устранило класс ошибок – делает вывод: *надо всегда добавлять такие проверки в будущем* (можно сохранить как правило или комментарий).

- **Цель:** Замкнуть цикл самосовершенствования. Теперь Ева не только реагирует на отдельную задачу, но и проактивно улучшает себя, опираясь на накопленный опыт.
- **Метрика:** *Результативность планов улучшения.* Из задач, предложенных ImprovementPlanner, сколько выполнено и привело ли к улучшению метрик. Например: Planner предложил 5 улучшений, из них 4 выполнены, 3 реально улучшили показатели – это хороший результат. Если большинство предложений либо не реализуется, либо не даёт эффекта – нужно улучшать анализ (возможно, привлекать LLM для более тонких выводов или настраивать пороги).

Шаг 7.3: Замыкание петли и автономная эволюция

- **Действия:** Объединить всё вместе:
    - DecisionEngine теперь на каждом шаге готов принимать задачи не только извне, но и от ImprovementPlanner. Сделать так, чтобы **внутренние цели** ставились автоматически. Например, после обработки обычной задачи DecisionEngine проверяет: есть ли предложения по самоулучшению? Если да, берет их как следующие задачи.
    - Обеспечить приоритеты: возможно, внешние задачи важнее, поэтому Ева сначала выполняет пользовательские запросы, а в паузах занимается самоулучшением. (На MVP можно последовательно, по мере появления).
    - Добавить в FeedbackRequester (если нужен) уведомления: например, если ImprovementPlanner решил улучшить что-то критичное, можно уведомить разработчика: «Ева собирается обновить свой модуль X для оптимизации – подтвердите». В идеале, Ева должна уметь и сама, но на MVP контроль не помешает.
    - Подумать о **границах самоизменений:** чтобы Ева не удалила что-то важное по ошибке, можно ввести ограничения (политики). Например: *не менять модуль DecisionEngine без явного подтверждения*, или *не удалять тесты*. Эти правила можно жестко зашить или хранить в памяти как "immutable parts" (в ReflectionLog или modules можно пометить «не трогать»).
- **Цель:** Ева 2.0 полностью замыкает цикл: она получает задачи, выполняет их, учится на результатах и меняет себя, что повышает её способность решать будущие задачи. Это базовая архитектура **самообучающегося ИИ**.
- **Метрика:** *Уровень автономности.* Можно постепенно уменьшать вмешательство человека и смотреть, насколько далеко Ева зайдёт самостоятельно:
    - Сколько улучшений она сделала без подсказки.
    - Сколько времени работает без человеческих правок.
    - Насколько сложные задачи берёт сама (по мере саморазвития, можно пробовать усложнять задания).

После этапов 5 и 6 совместно, минимально жизнеспособная система уже может выполнять простейший цикл самосовершенствования: например, пользователь дал задачу «оптимизируй функцию X» – Ева сгенерировала новый код, протестировала, исправила, внедрила. Этап 7 добавляет к этому стратегическое измерение: Ева

начинает *сама* решать, что ей улучшить дальше. Таким образом, мы видим зарождение **«зерна сознания»**: Ева обладает (пусть примитивным) самопониманием и целеполаганием для собственного развития.

Финальная архитектурная схема «Ева 2.0»

Ниже приведён обобщённый сценарий работы Евы 2.0, показывающий взаимодействие всех основных компонентов (модулей) и поток данных между ними – своего рода **«нейросхема»** системы:

1. **Постановка задачи:** Пользователь (через интерфейс API/CLI) передаёт новую задачу или цель в систему. Это сообщение поступает в **DecisionEngine** как входной сигнал.
   Пример: User -> **DecisionEngine**: *«Задача: добавить в функцию X обработку пустого ввода»*.

2. **Доступ к контексту:** DecisionEngine сначала обращается к **MemoryManager** за соответствующим контекстом из памяти. Он запрашивает данные о модуле X (из мета-карты: какая сейчас версия, какие есть метрики, известные проблемы) и историю связанных ошибок из ReflectionLog.
   Пример: **DecisionEngine** -> **MemoryManager**: *«Дай информацию о модуле X и последних ошибках, связанных с ним»* → **PostgreSQL** возвращает запись (метрики модуля X, отмечено, что не обрабатывает пустой ввод).

3. **Самоанализ перед действием:** DecisionEngine поручает **SelfMonitor** обновить мета-карту перед работой, чтобы начать с актуальных данных. SelfMonitor сканирует кодовую базу (или хотя бы модуль X и связанные) и обновляет **Code Map** в базе, включая текущие метрики.
   Пример: **DecisionEngine** -> **SelfMonitor**: *«Пересканируй модуль X»* → **SelfMonitor** парсит `module_x.py`, видит функции, пересчитывает сложность, обновляет запись в PostgreSQL.
   SelfMonitor также сообщает DecisionEngine, что, например, «Модуль X имеет сложность 15, покрытие тестами 50%, есть связь с модулем Y через импорт».

4. **Планирование:** DecisionEngine формирует план решения задачи. Если нужно, он использует **LLM Proxy** для помощи:
   Пример: **DecisionEngine** -> **LLM Proxy**: *«Как можно улучшить функцию X, чтобы обрабатывать пустой ввод?»* → **LLM Proxy** отправляет вопрос GPT-4 и получает ответ-план: «1) Добавить проверку if input is None: return или raise; 2) Добавить тест на пустой ввод». DecisionEngine фиксирует этот план.

5. **Генерация кода:** DecisionEngine маршрутизирует подзадачи плана к исполнителям:

- Шаг «Добавить проверку в функцию X» распознаётся как изменение существующего кода → DecisionEngine вызывает **CodeEditor**.
  DecisionEngine -> **CodeEditor**: *«В модуле X, в функции foo, добавь проверку: если входные данные пустые – возвращать ошибку.»*
  CodeEditor анализирует `module_x.py` (AST) и вносит соответствующую правку. При необходимости он может сам уточнить у GPT-4, как правильно внести изменение (особенно если изменение сложное). После успешного редактирования CodeEditor сохраняет файл.
- CodeEditor сообщает DecisionEngine, что модуль X изменён.

6. **Обновление мета-карты после изменения:** SelfMonitor автоматически (или по команде) пересканирует изменённый модуль и обновляет память:
   Пример: **SelfMonitor** -> **Code Map (DB)**: записывает, что в модуле X теперь появилась обработка пустого ввода, метрика «Простота» улучшилась, статус модуля X = «modified (needs test)».
   MemoryManager получает эти обновления.

7. **Генерация тестов:** DecisionEngine переходит к шагу «Добавить тест на пустой ввод». Он вызывает **TestGenerator**:
   DecisionEngine -> **TestGenerator**: *«Создай тест, который проверяет функцию X на пустой ввод»*.
   TestGenerator через LLM Proxy генерирует код теста (например, `test_module_x.py` со случаем вызова foo(None) и ожиданием исключения), сохраняет его.

8. **Запуск тестов в песочнице:** DecisionEngine вызывает **TestRunner** для запуска тестов изолированно:
   DecisionEngine -> **TestRunner**: *«Выполни тесты для модуля X в Docker»*.
   TestRunner -> **Sandbox (Docker)**: собирает код (обновлённый модуль X и новый тест), запускает контейнер, внутри которого выполняется pytest.
   Sandbox возвращает результаты: например, «1 тест выполнен, **пройден успешно**» (или неуспешно с логом ошибки).
   TestRunner возвращает DecisionEngine отчёт: *все тесты прошли*.

9. **Анализ результатов:** (В данном примере тесты прошли, но рассмотрим и вариант ошибки):

   - **Если тесты прошли:** DecisionEngine помечает изменение успешным. Он сообщает SelfMonitor, что модуль X теперь обновлён и **активен** (обновляет статус health = OK в базе). Можно зафиксировать новую версию в `code_versions`.
   - **Если тесты не прошли:** DecisionEngine вызывает **TestAnalyzer** для диагноза:
     DecisionEngine -> **TestAnalyzer**: *«Проанализируй, почему упал тест»*.
     TestAnalyzer получает из Sandbox лог (например, AssertionError на ожидаемое исключение). Через LLM Proxy (или по шаблону) определяет, что код не бросает исключение, а молча возвращает None. Рекомендует: «В функции X вместо `return` None нужно генерировать исключение

ValueError».
TestAnalyzer -> **DecisionEngine**: *«Диагноз: функция X не соответствует ожиданиям – нужно бросать ValueError при пустом вводе.»*.
DecisionEngine решает применить эту рекомендацию: снова вызывает **CodeEditor** с инструкцией внести правку (генерировать исключение).
Далее цикл повторяется: SelfMonitor обновляет код -> TestGenerator/Runner заново тестируют -> и так до успеха или предела.

10. **Завершение задачи:** В случае успеха DecisionEngine формирует ответ пользователю: *«Модуль X оптимизирован: добавлена обработка пустого ввода.»*. Он также логирует это событие в ReflectionLog (с отметкой урока, если был: например, «Научилась, что нужно учитывать None в таких-то случаях»).
Если были неудачные попытки или промежуточные уроки (например, "попытка 1: возвращать None оказалась неправильной"), они тоже записаны.

11. **Обновление знаний и замыкание цикла: ImprovementPlanner** получает доступ к свежим записям:

    - Из ReflectionLog: видит, что проблема с пустым вводом решена после 2 попыток.
    - Из мета-карты: видит, что сложность модуля X снизилась, покрытие тестами выросло (был добавлен тест).
      ImprovementPlanner может на будущее добавить правило: *«при разработке функций проверять None сразу»*. И/или отметить, что модуль X теперь в норме.
      Он также проверяет, остались ли другие повторяющиеся проблемы, и может сформировать новую цель.
      Пример: **ImprovementPlanner** -> **DecisionEngine**: *«Предлагаю улучшить модуль Y, там 3 раза были таймауты»*.

12. **Новая внутренняя цель:** DecisionEngine принимает цель от ImprovementPlanner (может уведомить разработчика, если настроено) и снова проходит цикл планирования → генерации → тестирования для этой цели.
Таким образом, после внешней задачи Ева может автоматически заняться своими внутренними улучшениями.

Этот цикл повторяется постоянно, обеспечивая самоподдерживающееся развитие системы. Ниже схематично показаны основные взаимодействия (для краткости не все модули указаны):

Пользователь -> [DecisionEngine] -> MemoryManager -> PostgreSQL (получение контекста)
DecisionEngine -> SelfMonitor -> (обновление мета-карты) -> PostgreSQL
DecisionEngine -> LLM Proxy -> GPT-4 (получение плана/кода)
DecisionEngine -> CodeGenerator/CodeEditor -> LLM Proxy (генерация/правка кода)
CodeGenerator/Editor -> Code Map (обновление кода) -> SelfMonitor -> PostgreSQL
DecisionEngine -> TestGenerator -> LLM Proxy (генерация тестов)
DecisionEngine -> TestRunner -> Docker Sandbox (выполнение тестов)
Docker Sandbox -> TestRunner -> DecisionEngine (результаты тестов)

DecisionEngine -> TestAnalyzer -> LLM Proxy (анализ ошибок) -> DecisionEngine (диагноз)
alt [если ошибки]:
  DecisionEngine -> CodeEditor (исправление по диагнозу) -> ... цикл повторяется ...
else [если успех]:
  DecisionEngine -> SelfMonitor/MemoryManager (пометить код как здоровый) -> PostgreSQL
  DecisionEngine -> Пользователь (ответ о успехе)
DecisionEngine -> ReflectionLog (запись опыта)
ImprovementPlanner -> ReflectionLog/DB (анализ опыта и метрик)
ImprovementPlanner -> DecisionEngine (новая цель улучшения)

*(Примечание: эта текстовая схема эквивалентна PlantUML-диаграмме, описанной в документации. Она демонстрирует, как **«синапсы»** – стрелки взаимодействия – связывают все компоненты в единый цикл.)*

Заключение: Минимально жизнеспособная версия «Ева 2.0» теперь представляет собой комплексную систему, в которой реализованы все основные аспекты: **модульность** (кубики с чёткими ролями), **память** (централизованная база знаний о себе), **самонаблюдение** (мета-карта кода и метрики), **генерация и модификация кода** (с помощью LLM, под контролем алгоритмов), **тестирование в изоляции** (Docker-песочница) и **петля самообучения** (журнал опыта и улучшения).

Этот фундамент позволяет Еве не только автоматизировать программирование под заданные задачи, но и постепенно **улучшать саму себя**, приближаясь к идее «живого» ИИ. В дальнейшем, по мере накопления опыта, можно усложнять цели, наращивать число кубиков (для более тонких функций), вводить механизмы приоритизации памяти (забывания), подключать свои модели вместо API – архитектура уже рассчитана на масштабирование без ломки основных принципов. Каждый новый «нейрон» или «синапс» впишется в существующую карту.

Метрики успеха всей системы на уровне MVP можно резюмировать так:

- *Автономность:* какой процент задач Ева решает без помощи (стремиться увеличивать);
- *Качество кода:* средние метрики (сложность, покрытие) со временем улучшаются или хотя бы не деградируют;
- *Обучаемость:* скорость, с которой уменьшается количество итераций для схожих задач (если раньше на исправление уходило 3 попытки, а теперь, научившись, 1–2);
- *Стабильность:* Ева не разрушается от собственных изменений (ни один успешный функционал не сломан после цикла улучшений – регрессий нет).

Выполнив все этапы, мы получаем работающую MVP-версию «Евы 2.0», готовую к экспериментам и дальнейшему развитию. Ее архитектура уже сейчас отражает принципы, приближенные к биологическим: раздельные специализированные

«области мозга», нейрообразные связи (с возможностью гибкой перенастройки), и цикл рефлексии, благодаря которому система может расти интеллектуально. Это первый шаг к по-настоящему эволюционирующему ИИ.